

# NCA Sponsored Training

Day 2 – Morning Session (Theory)

## Password Cracking

*Ahmed Didouh*

*Madinah*

*13/01/2026*



[rc3.kaust.edu.sa](http://rc3.kaust.edu.sa)





# Outline



## 1. Introduction to Password

- *What is a Password*
- *Password Management*
- *Common Attacks*

## 2. Hash Functions

- *Introduction*
- *Hash Function for Password Storage*

## 3. Hash Functions for Secure Password Storage



## What is a password ?

- It is an arbitrary long sequence of characters
- It should be secret
- It should be difficult to guess





## Basic Problem

- How do you prove to someone that you are who you claim to be?
- Any system with access control must solve this problem

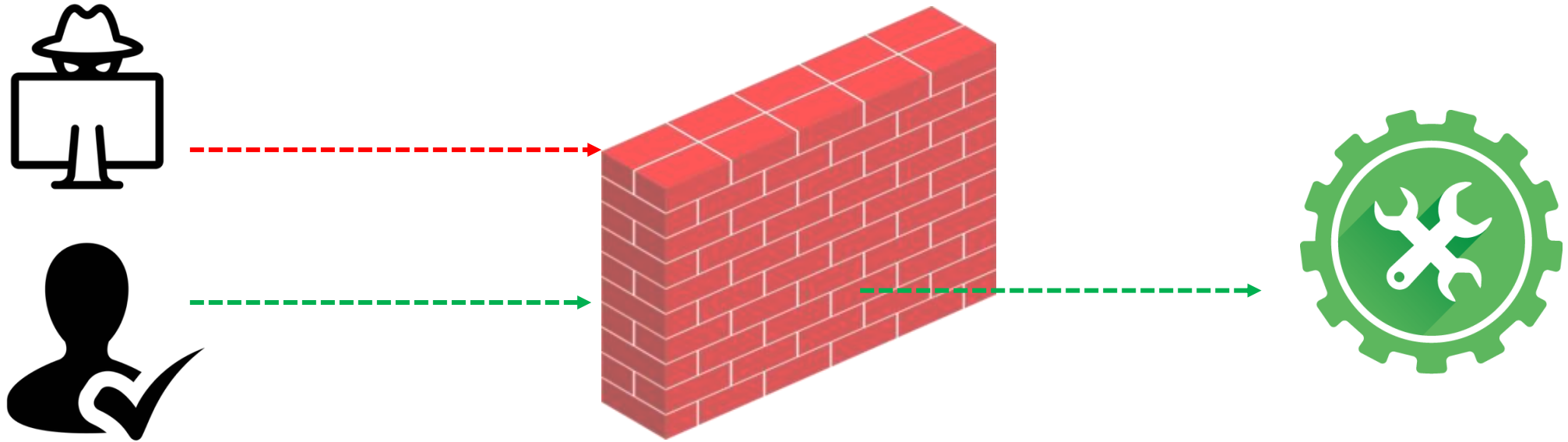




# Passwords



Why we need passwords ?





# Passwords



## Authentication

- User has a secret password.
- System checks it to authenticate the user.

Vulnerable to **eavesdropping** when password is communicated from user to system





## Basic Security problems

- How easy is it to guess the password?
- How is the password stored?
- How does the system check the password?





# Passwords – Guess Difficulty



How easy is it to guess the password?

- A lock constituted by three cylinders... only digits!
- In order to open it, you need to find the right sequence

| X | Y | Z |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 1 | 1 |
| 2 | 2 | 2 |
| 3 | 3 | 3 |
| 4 | 4 | 4 |
| 5 | 5 | 5 |
| 6 | 6 | 6 |
| 7 | 7 | 7 |
| 8 | 8 | 8 |
| 9 | 9 | 9 |

What if you do not know it ?

- The three digits sequence opening the lock
- should be chosen among  $10 \times 10 \times 10 = 1000$

**One out of 1000 !**





# Passwords – Guess Difficulty



**Let's try to break a key of 6 digits ...**

Pick Me! I'll do it



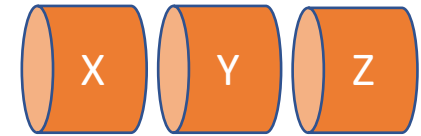


# Passwords – Guess Difficulty



A lock constituted by three cylinders... **not just digit!**

In order to open it, you need to find the right sequence



Each of the cylinder has  $10 + 26 = 36$  positions:

- 1234567890 [10]
- qwertyuiopasdfghjklzxcvbnm[26]

The guessing difficulty goes up to:

$$36 * 36 * 36 = 46656$$



# Passwords – Guess Difficulty



Passwords are usually constituted by (at least) about 8 characters



Each of the cylinder has  $10 + 26 = 36$  positions:

- 1234567890 [10]
- qwertyuiopasdfghjklzxcvbnm[26]

**Is this enough ?**

The guessing difficulty goes up to:

$$36 * 36 * 36 * \dots * 36 = 2821 \text{ Billions}$$



# Passwords – Brute Force Attack



Brute force attack consists of **systematically** try all the combinations of characters

- Characters are more than 26 + 10
- We should also count for punctuation... about 10 more (the most used)
- For each character we have 26 + 10 + 10 = **46 different possibilities**

Brute-forcing a password of 10 characters will take time:

- $46^{10} = 42,420,747,482,776,576$  (~42 Millions of Billions)



## Is this enough ?



# Brute Force



Nvidia's new **RTX 4090** graphics card is powerful enough to break password-cracking records, according to benchmarks by a password recovery firm.

Tested against:

- Microsoft's New Technology LAN Manager (NTLM) authentication protocol: **300GH/sec**
- Bcrypt: **200KH/sec**

**KH/s** (kilo): thousand hashes per second

**GH/s** (giga): billion hashes per second





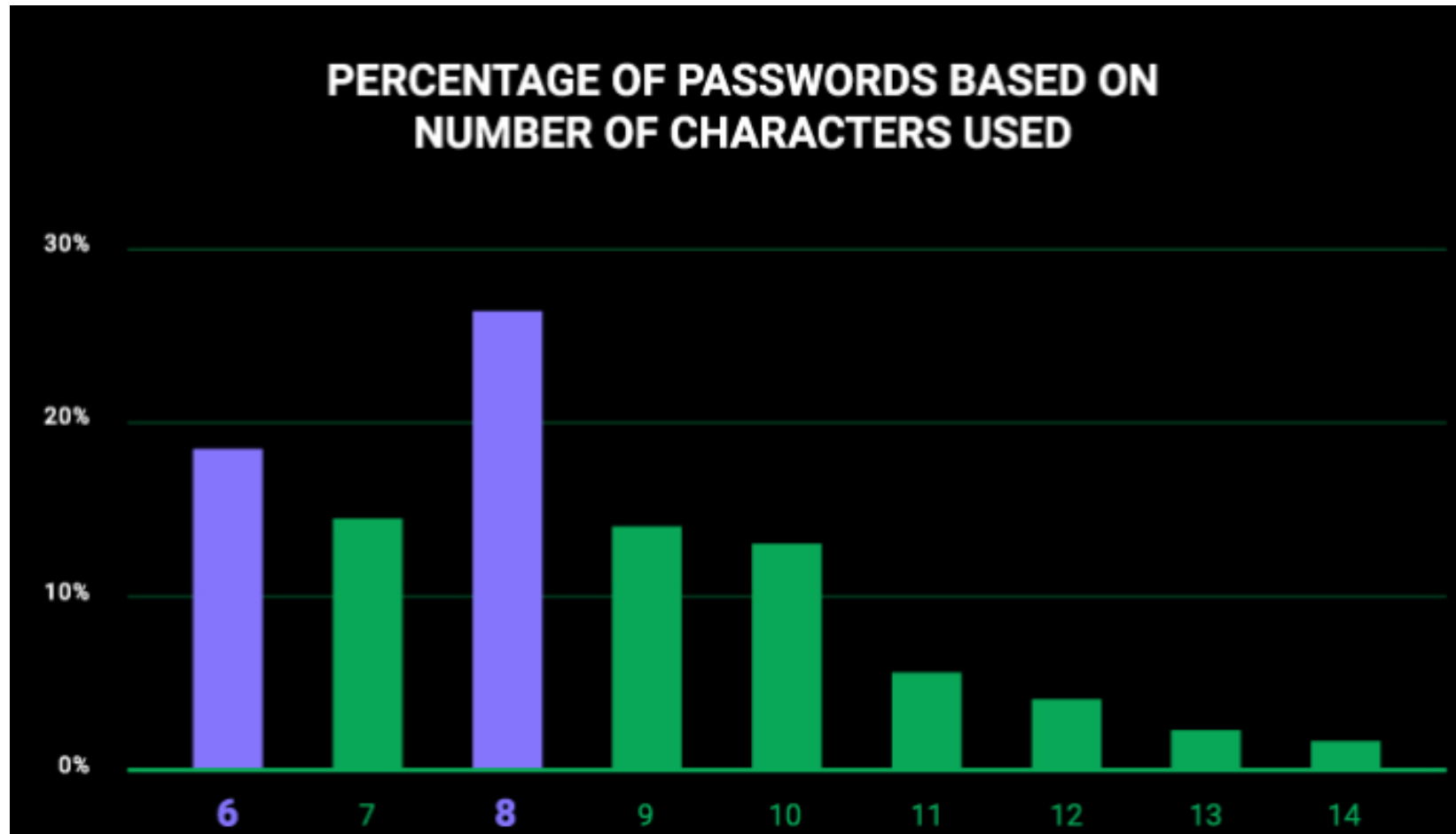
# Passwords – How Secure is your Password?



| Number of Characters | Numbers Only | Lowercase Letters | Upper and Lowercase Letters | Numbers, Upper and Lowercase Letters | Numbers, Upper and Lowercase Letters, Symbols |
|----------------------|--------------|-------------------|-----------------------------|--------------------------------------|-----------------------------------------------|
| 4                    | Instantly    | Instantly         | Instantly                   | Instantly                            | Instantly                                     |
| 5                    | Instantly    | Instantly         | Instantly                   | Instantly                            | Instantly                                     |
| 6                    | Instantly    | Instantly         | Instantly                   | 1 sec                                | 5 secs                                        |
| 7                    | Instantly    | Instantly         | 25 secs                     | 1 min                                | 6 mins                                        |
| 8                    | Instantly    | 5 secs            | 22 mins                     | 1 hour                               | 8 hours                                       |
| 9                    | Instantly    | 2 mins            | 19 hours                    | 3 days                               | 3 weeks                                       |
| 10                   | Instantly    | 58 mins           | 1 month                     | 7 months                             | 5 years                                       |
| 11                   | 2 secs       | 1 day             | 5 years                     | 41 years                             | 400 years                                     |
| 12                   | 25 secs      | 3 weeks           | 300 years                   | 2k years                             | 34k years                                     |
| 13                   | 4 mins       | 1 year            | 16k years                   | 100k years                           | 2m years                                      |
| 14                   | 41 mins      | 51 years          | 800k years                  | 9m years                             | 200m years                                    |
| 15                   | 6 hours      | 1k years          | 43m years                   | 600m years                           | 15 bn years                                   |
| 16                   | 2 days       | 34k years         | 2bn years                   | 37bn years                           | 1tn years                                     |
| 17                   | 4 weeks      | 800k years        | 100bn years                 | 2tn years                            | 93tn years                                    |
| 18                   | 9 months     | 23m years         | 6tn years                   | 100 tn years                         | 7qd years                                     |



# Passwords – How Secure is your Password?





# Password Strength



**First issue: How do you chose your password?**

Passwords are never chosen like the following:

- j39dh5ks
- d93kdh3a
- k30fj50d



The above are 8 **random characters** passwords.

Your passwords do not look like the above ones, right ?





# Password Strength



**Usually, passwords contain:**

- Personal information (names/surnames, phone numbers, birthdate, etc.)
- Sport teams and sport terms
- Cities
- Months, weekdays, seasons
- Food
- Keyboard pattern

**This habit significantly decreases the difficulty to guess your passwords**

**... and the bad guy will be very happy !!!**





# Top 20 most common passwords of 2023



- |                |                 |
|----------------|-----------------|
| 1) 123456      | 11) UNKNOWN     |
| 2) Admin       | 12) 1234567     |
| 3) 12345678    | 13) 123123      |
| 4) 123456789   | 14) 111111      |
| 5) 1234        | 15) Password    |
| 6) 12345       | 16) 12345678910 |
| 7) Password    | 17) 000000      |
| 8) 123         | 18) *****       |
| 9) Aa123456    | 19) Aa123456    |
| 10) 1234567890 | 20) user        |



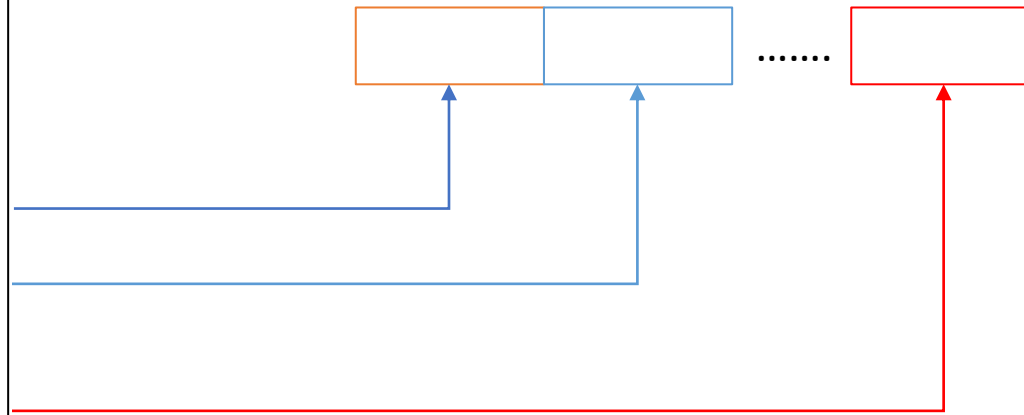


# Base-Words



saudi  
ahmed  
sara  
hamad  
riyadh  
nasser  
vision  
khalid  
omar  
aziz  
assan  
reem  
noor  
aisha  
...

Base-words are words referring to concepts that can be combined to build passwords



Example: **vision2030, qwertysaudi, omar1981**



## Base-Words Let's count



The English dictionary contains about 170,000 words.

Let us assume you combine:

- **One word + date (4 digits) =  $170,000 * 10,000 = 17 \text{ Billions}$**
- **Two words =  $170,000 * 170,000 = 29 \text{ Billions}$**

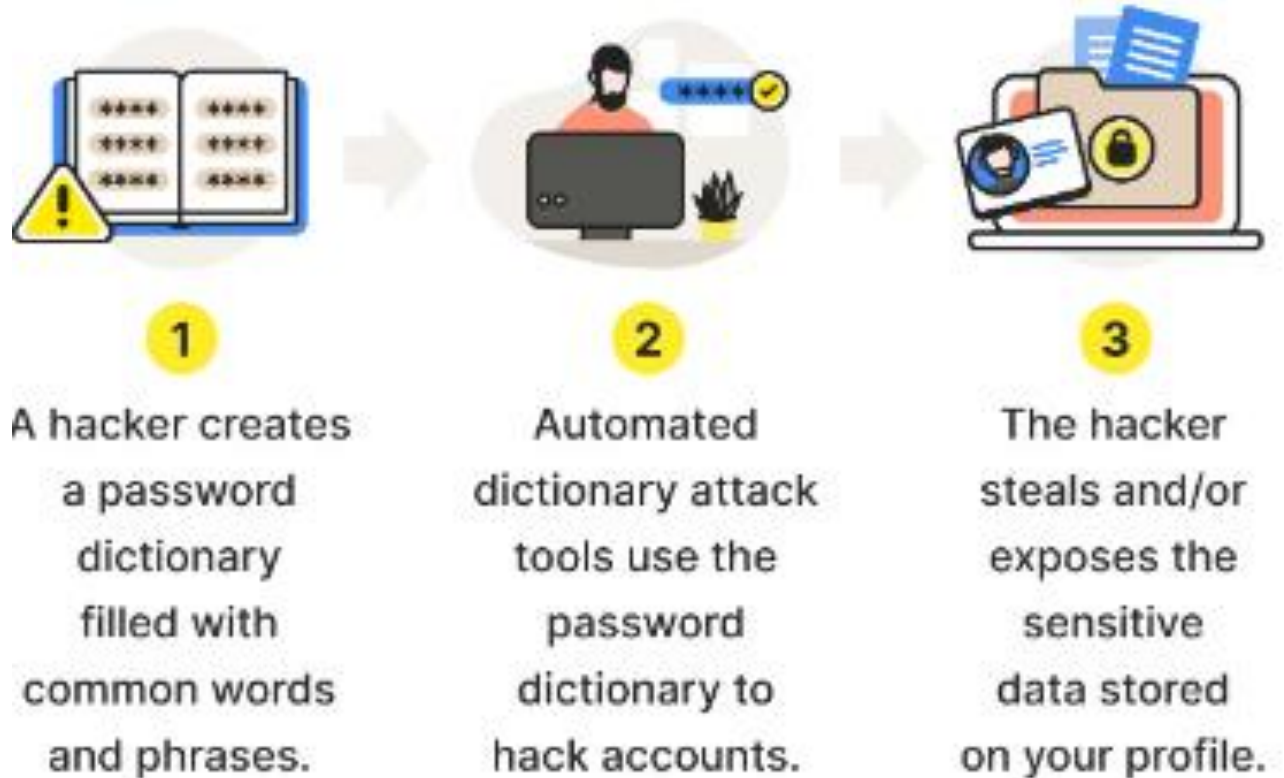
**An easy game for the RTX 4090...**



# Dictionary Based Attack



- The smart adversary collects the passwords from previous database leaks
- and re-use them (or combine them) to increase his chances to break in





# Authentication – Basic Principles



Can we mitigate password attacks with better authentication systems?

- Knowledge factors (**what you know**) include all things users must know to access the system. Usernames, IDs, passwords and personal identification numbers (PINs) all fall under this category.
- Possession factors (**what you have**) consist of anything users must have in their possession to log in. This category includes one-time password tokens, key fobs, smartphone apps, and employee ID cards.
- Inherence factors (**what you are**) include characteristics inherent to individuals that confirm their identity. This category includes the scope of biometrics, such as retina scans, fingerprint scans, facial recognition and Voice authentication.

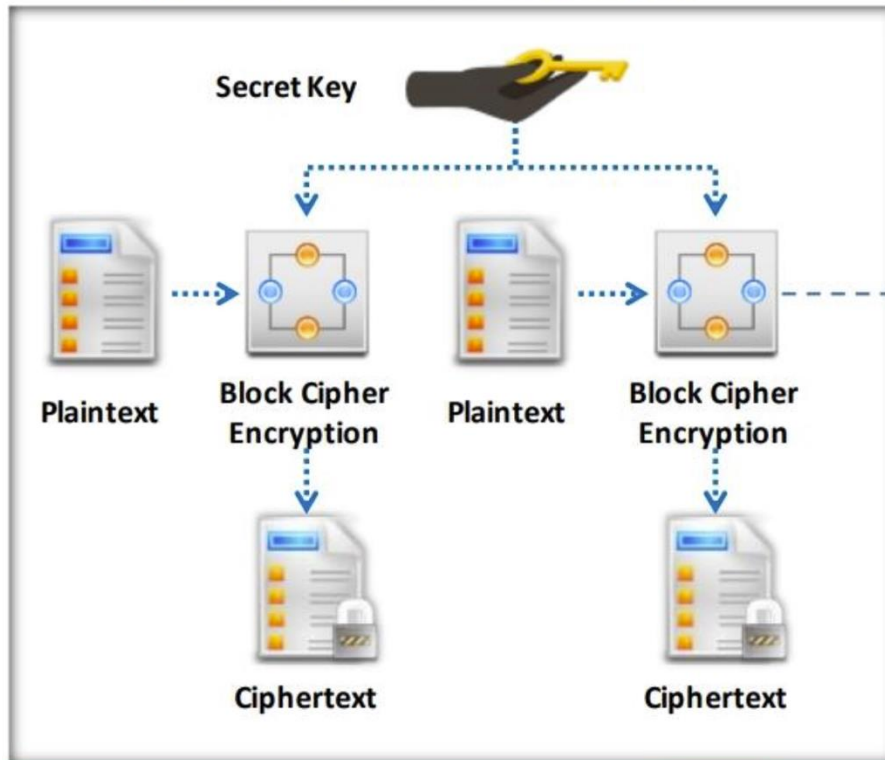


# Cipher Modes of Operation

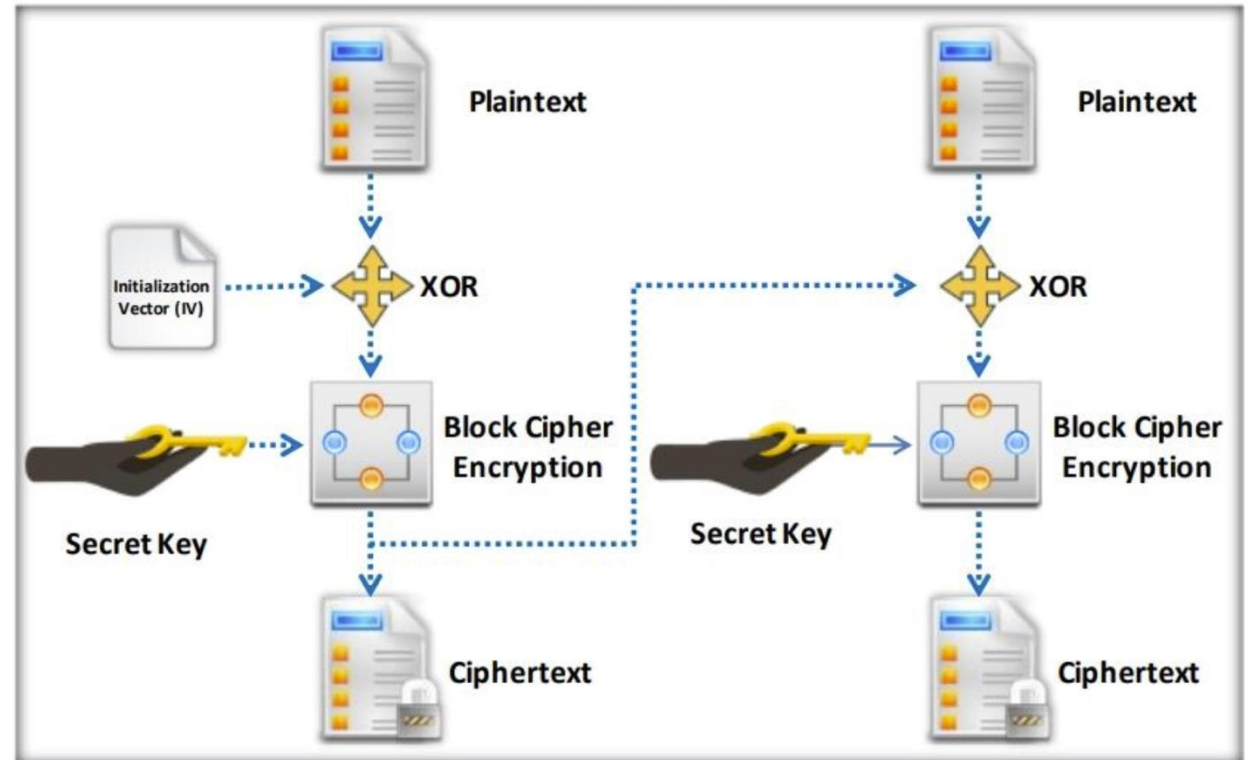


- Cipher modes of operations are used to **encrypt a fixed block of plain text** using secret key and Initialization vector
- These modes of operations can provide data with confidentiality and authenticity

**Electronic Code Book (ECB) Mode**



**Cipher Block Chaining (CBC) Mode**

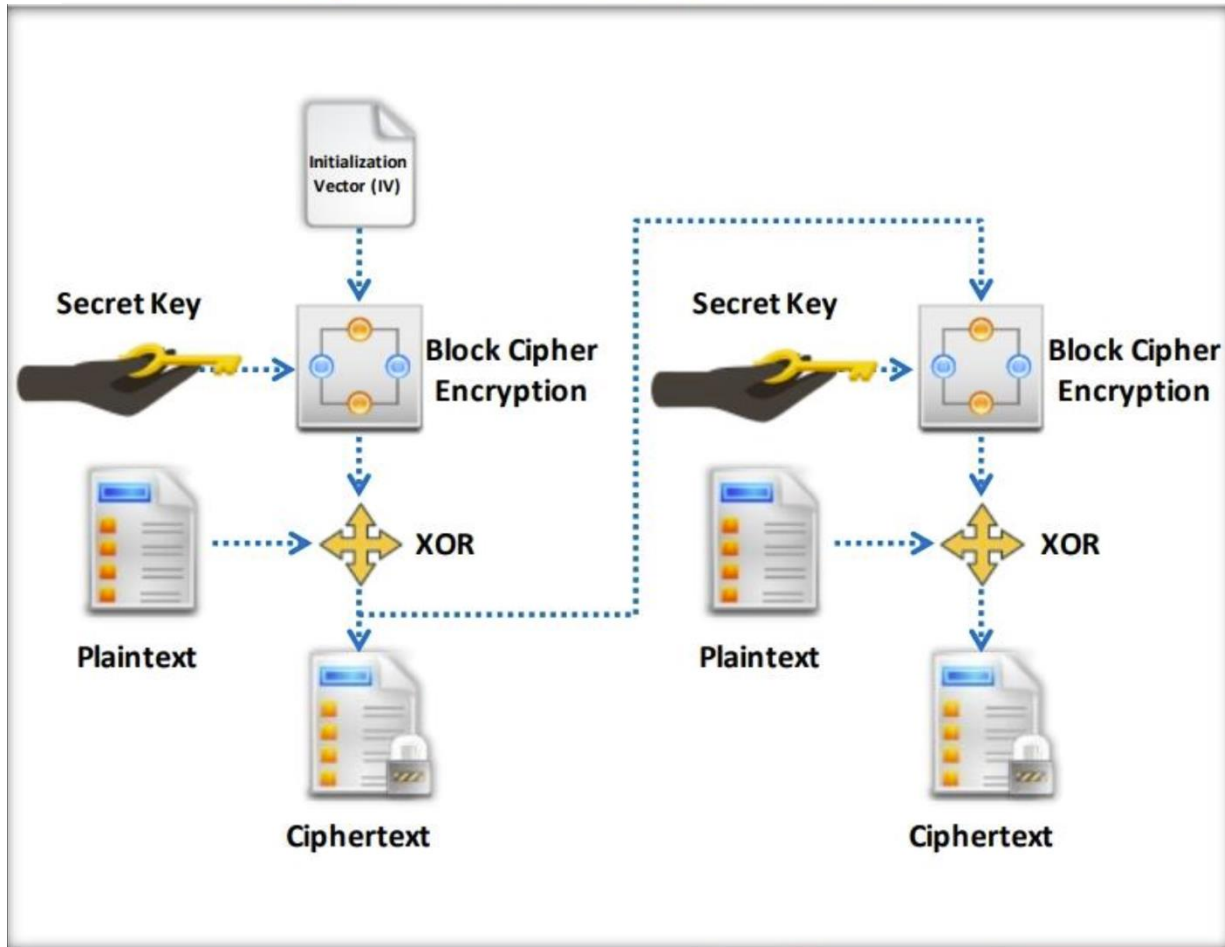




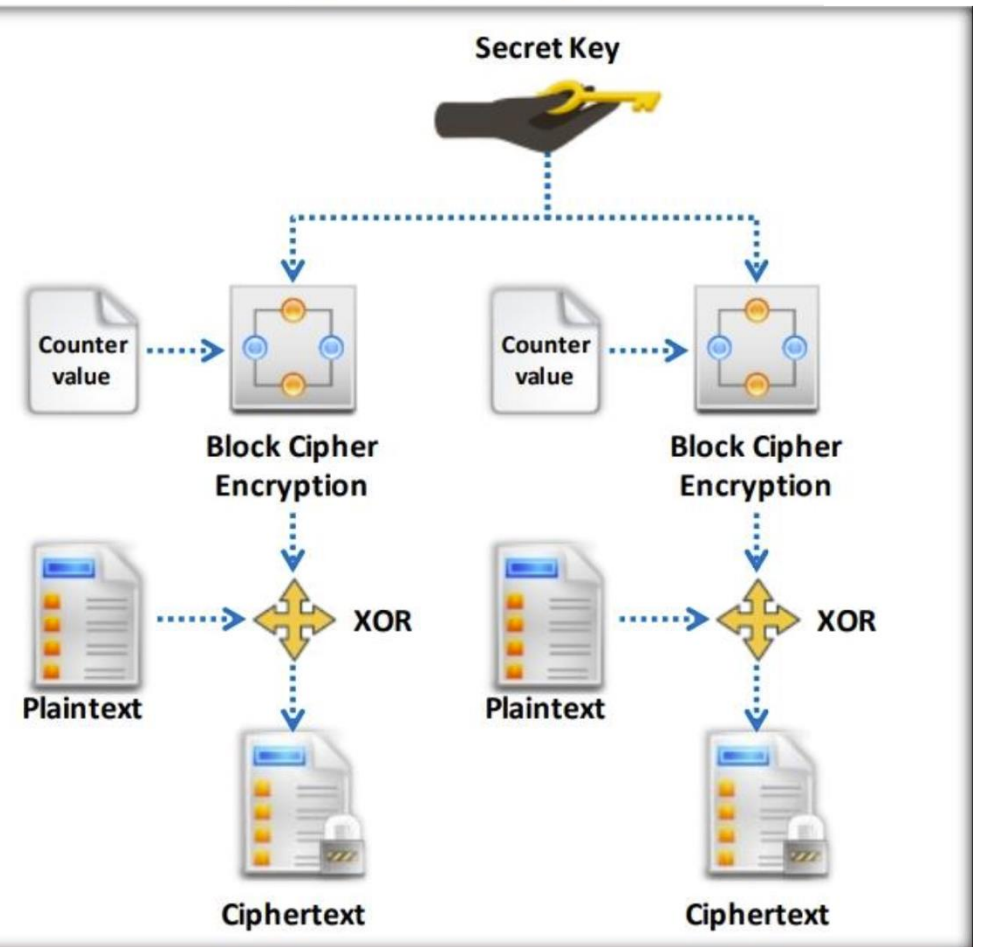
# Cipher Modes of Operation (Cont'd)



## Cipher Feedback (CFB) Mode



## Counter Mode







# Modes of Authenticated Encryption



- Modes of authenticated encryption ensure the **integrity** and **confidentiality** of transmitted messages and rectifies the problem of chosen ciphertext attacks

## Authenticated Encryption with Message Authentication Code (MAC)

- The message authentication code (MAC) is a value obtained by **hashing a plaintext message** using a shared secret key
  - Encrypt-then-MAC (EtM)
  - Encrypt-and-MAC (E&M)
  - MAC-then-encrypt (MtE)

## Authenticated Encryption with Associated Data (AEAD)

- AEAD is another approach used to ensure the **integrity** and **authenticity** of a message
- This approach adds additional data to the ciphertext at certain places to **thwart chosen ciphertext attacks**

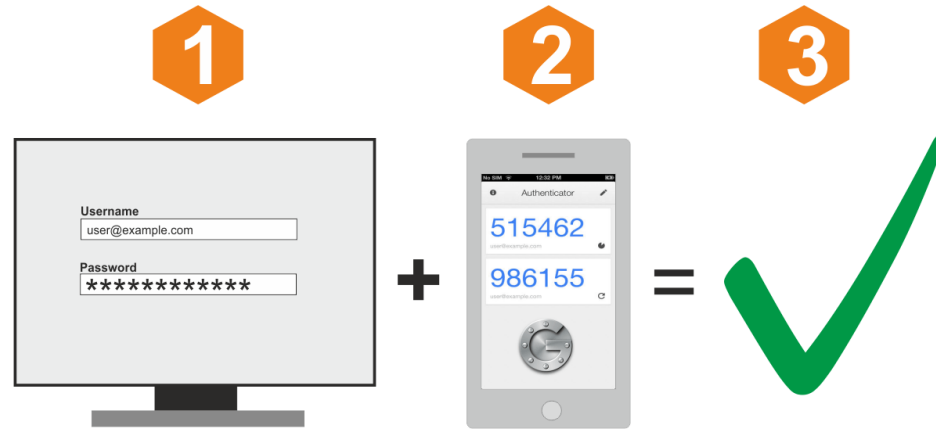




# Two Factor Authentication



**The user is granted the access to a system only after presenting different evidences that authenticate himself**



Standard two factor authentication systems rely on:

- Password
- Mobile phone (SMS/email)



# Two Factor Authentication



**Standard two factor authentication systems rely on:**

- Password
- Mobile phone SMS/email

**Login is performed by:**

- Entering the secret password
- Entering a PIN sent to the user's SMS

**The adversary has to:**

- Compromise the password
- Steal your mobile-phone

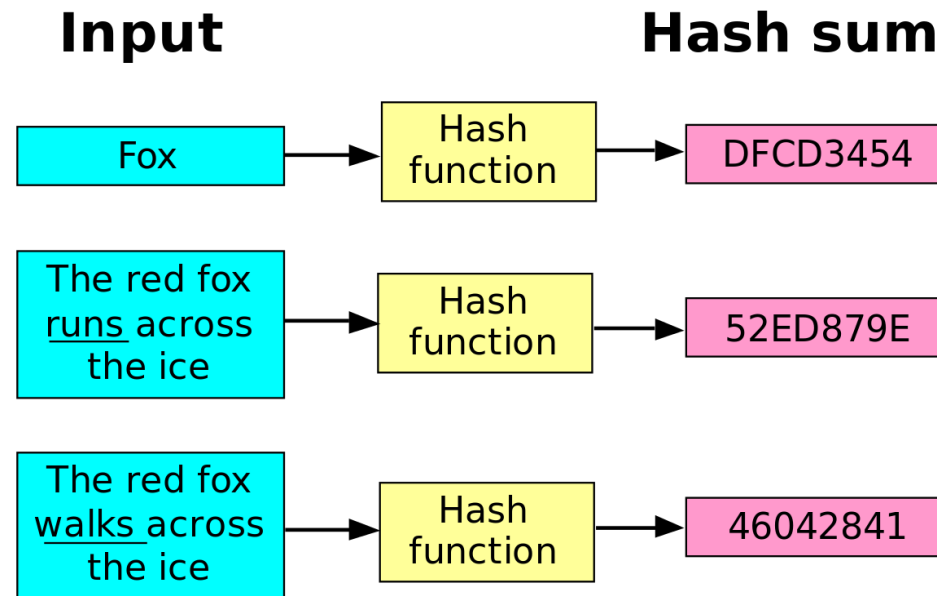




# Introduction to Hash Functions



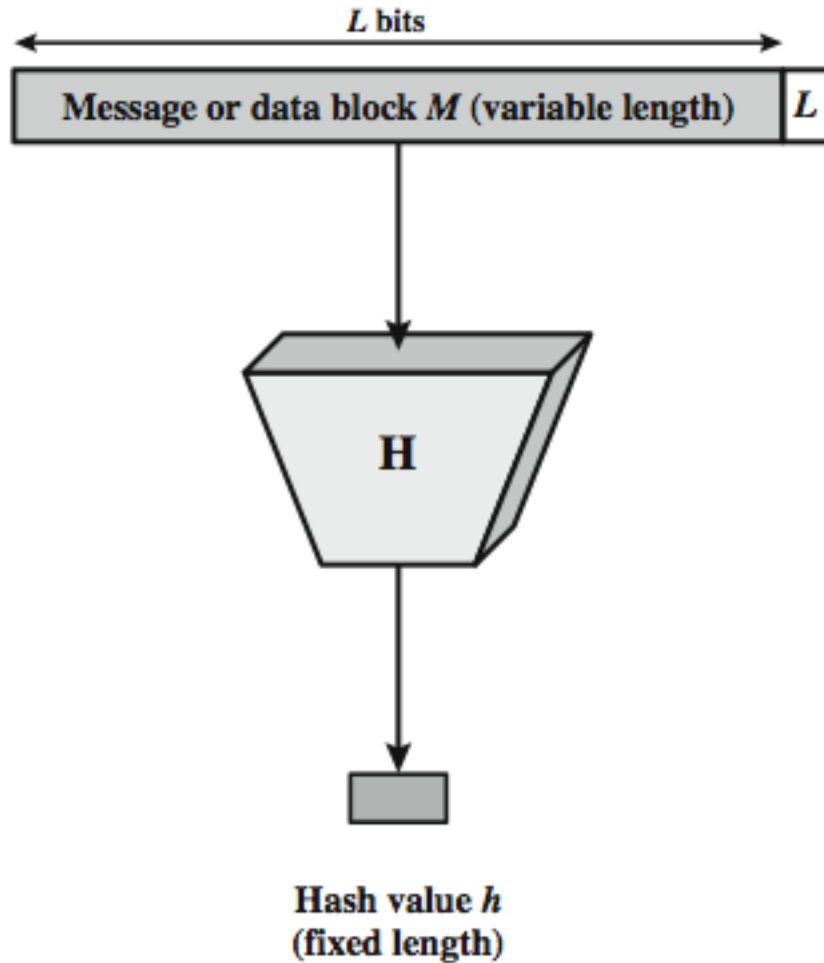
- A **hash function** is a function used to map data of **arbitrary** size to data of a **fixed** size.



- Example of hashing functions are: **MD5, RIPEMD-160, SHA-1, SHA-2, SHA-3**



# Introduction to Hash Functions



- The hash value represents concisely the longer message
  - may called the *message digest*
- A message digest is as a ``digital fingerprint'' of the original document

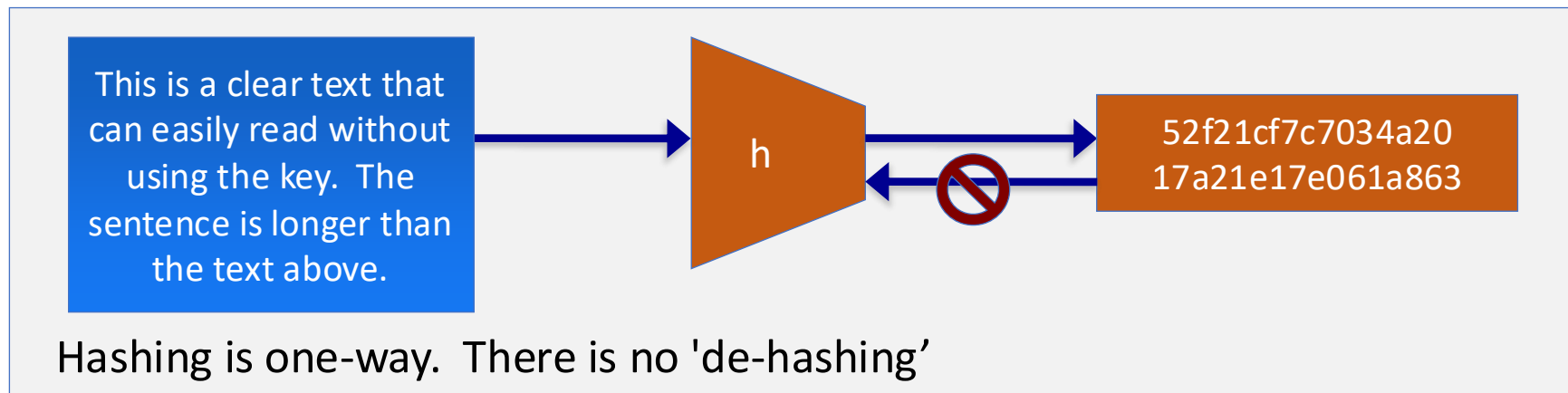
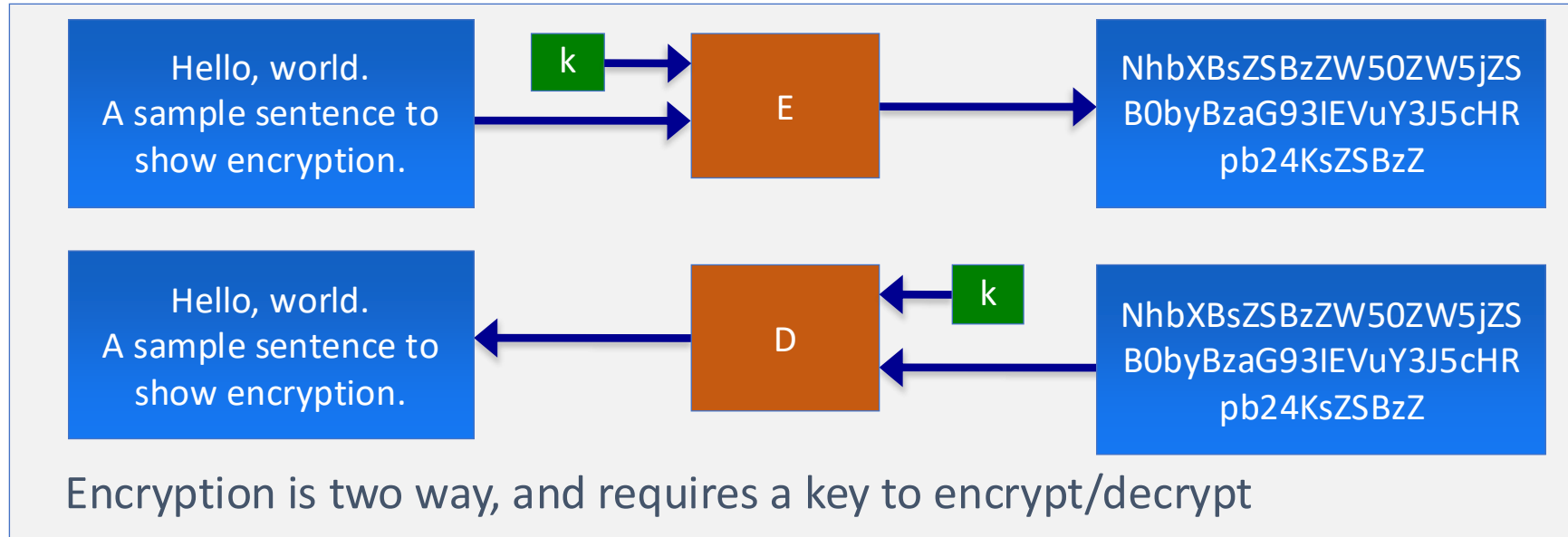


condenses arbitrary message to fixed size

$$h = H(M)$$



# Hashing V.S. Encryption





# Hash Function Applications



## Used Alone

- **Fingerprint** -- file integrity verification, public key fingerprint
- **Password storage** (one-way encryption)

## Combined with encryption functions

- **Hash based Message Authentication Code (HMAC)**
  - protects both a message's integrity and confidentiality
- **Digital signature**
  - Ensuring Non-repudiation
  - Encrypt hash with private (signing) key and verify with public (verification) key



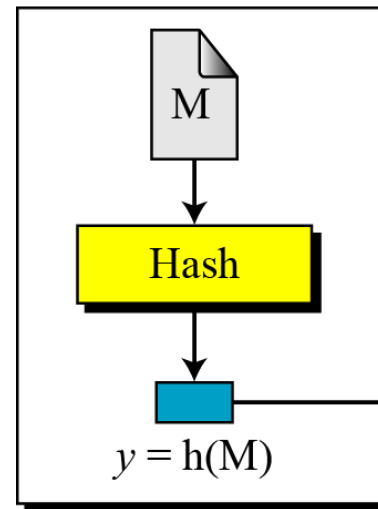
# Hash Functions – Security Requirements



## preimage resistant (one-way):

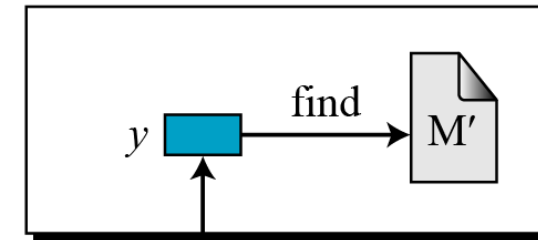
- the hash function must be one-way
- Given only a message digest, can't find any message (or preimage) that generates that digest.

M: Message  
Hash: Hash function  
 $h(M)$ : Digest



Alice

Given:  $y$   
Find: any  $M'$  such that  
 $y = h(M')$



Eve

To Bob



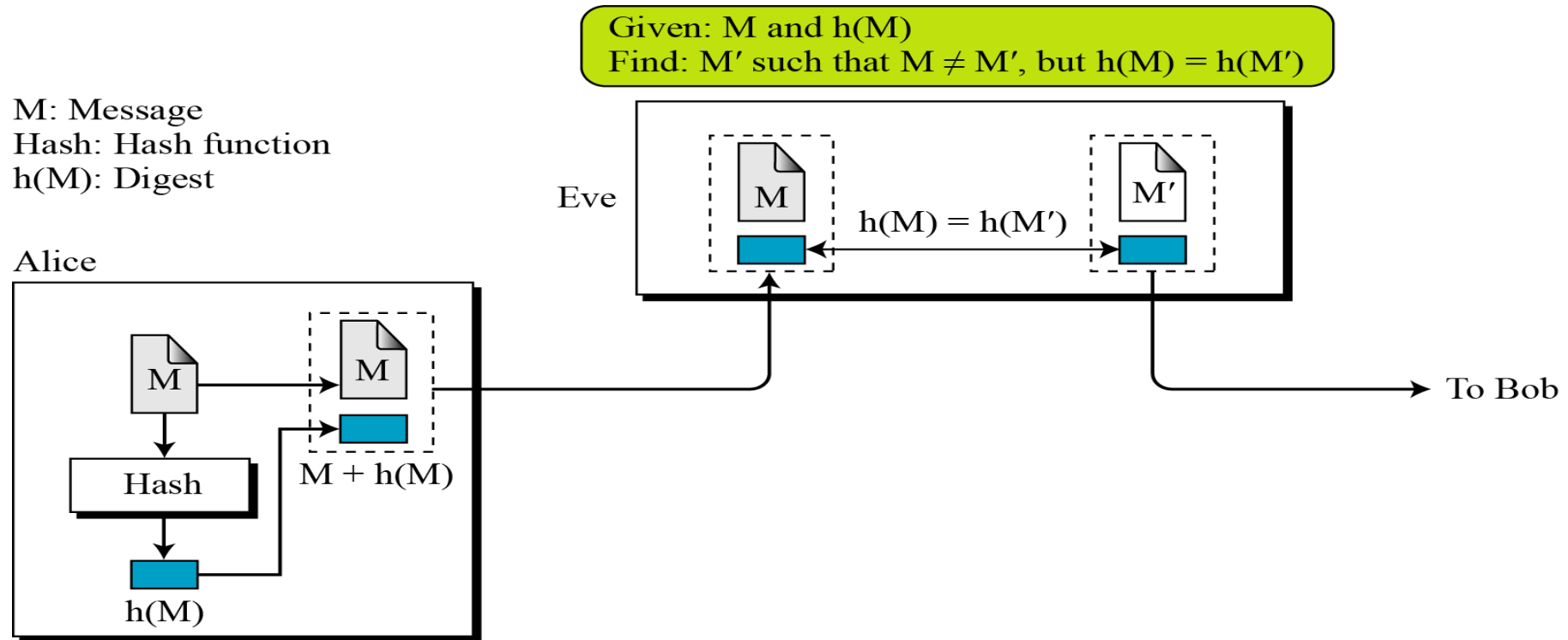


# Hash Functions – Security Requirements



## 2-nd preimage resistant (weak collision resistant):

- Given one message, can't find another message that has the same message digest.





# Hash Functions – Security Requirements



## Collision resistant (strong collision resistant):

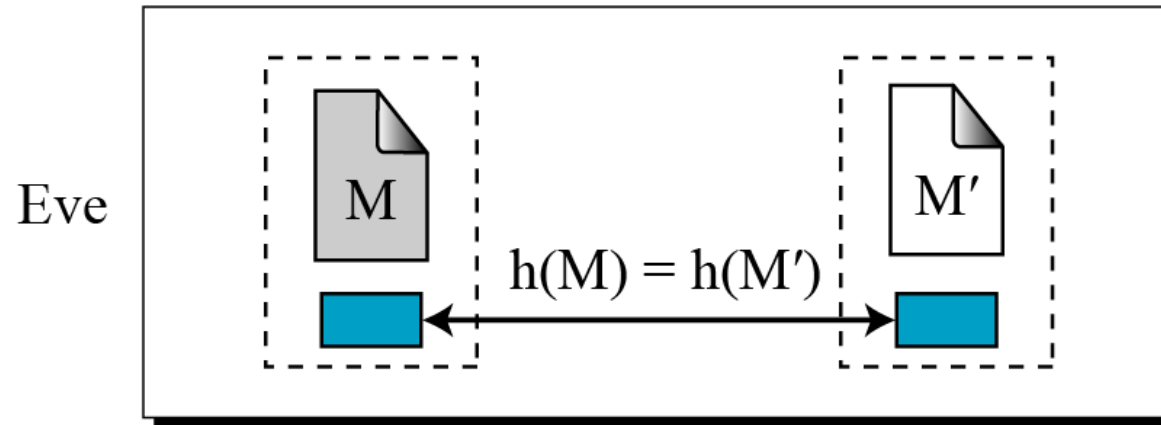
- Can't find any two different messages with the same message digest
- Collision resistance implies second preimage resistance

M: Message

Hash: Hash function

$h(M)$ : Digest

Find: M and M' such that  $M \neq M'$ , but  $h(M) = h(M')$





# Well Known Hash Functions



- **MD5**
  - output 128 bits
  - collision resistance completely broken by researchers in China in 2004
- **SHA1**
  - output 160 bits
  - no collision found yet, but method exist to find collisions in less than  $2^{80}$
  - considered insecure for collision resistance
  - one-wayness still holds

*"md5 and sha1 are both clearly broken (in terms of collision-resistance)".*

Ron Rivest



# Well Known Hash Functions



- **SHA2** (SHA-224, SHA-256, SHA-384, SHA-512)
  - outputs 224, 256, 384, and 512 bits, respectively (hash length)
  - SHA-256 is considered one of the most secure hashing functions. Even so, it is used in most SSL certificates and even crypto transactions.
  - Supported by the majority of browsers we are using today
  - No real security concerns (so far)
- **SHA3** (SHA3-224, SHA3-256, SHA3-384, SHA3-512)
  - based on an innovative cryptographic system called Sponge System
  - relatively fewer implementation costs
  - much faster than the previous versions
  - Ready to replace SHA2 when needed (the NSA has not asked to start using it to replace SHA2)



# Hashing Password



- Instead of user password, store hash of password
- When user enters password, compute its hash and compare with entry in password file

**System does not store actual passwords!**

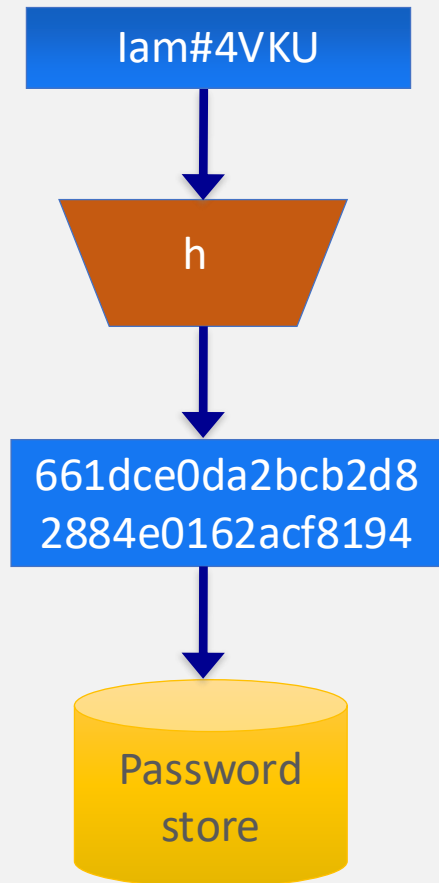




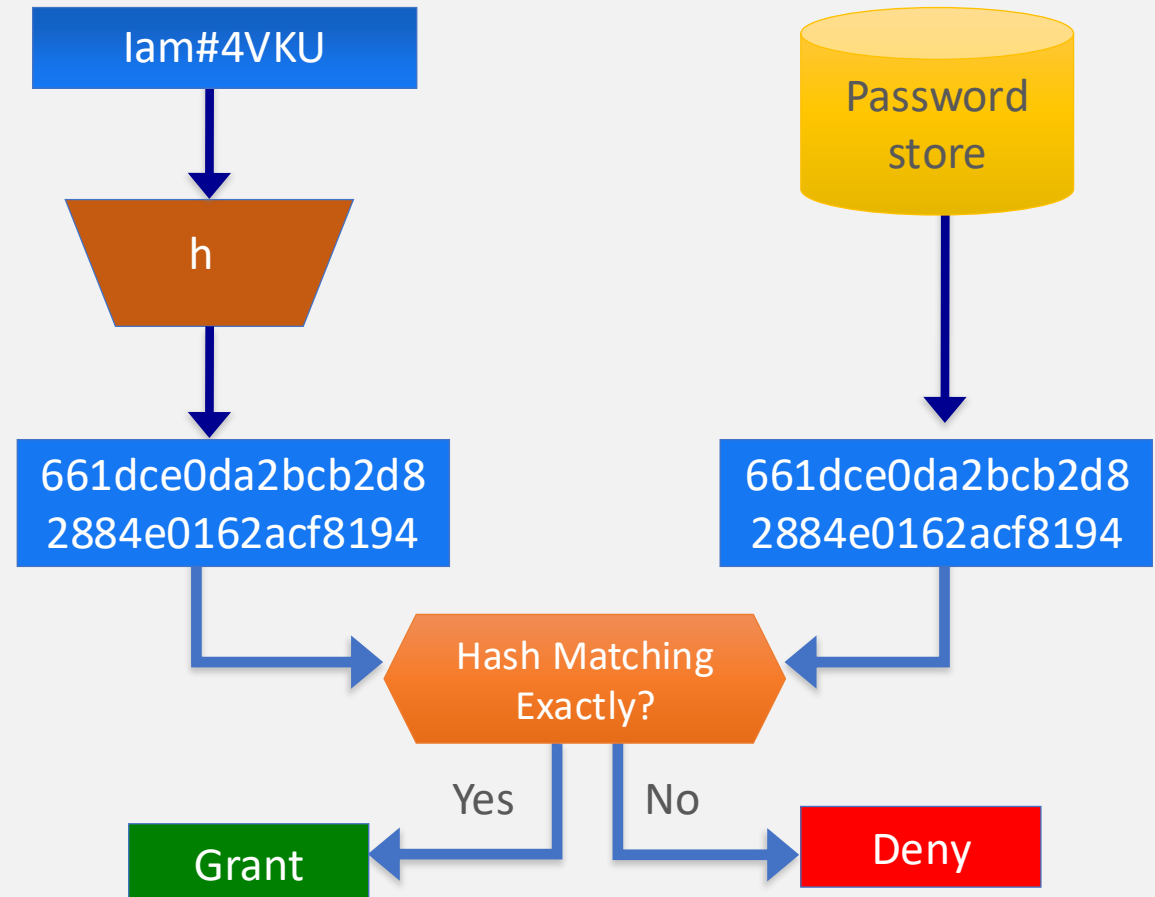
# Hashing Password



## Store Hashing Password

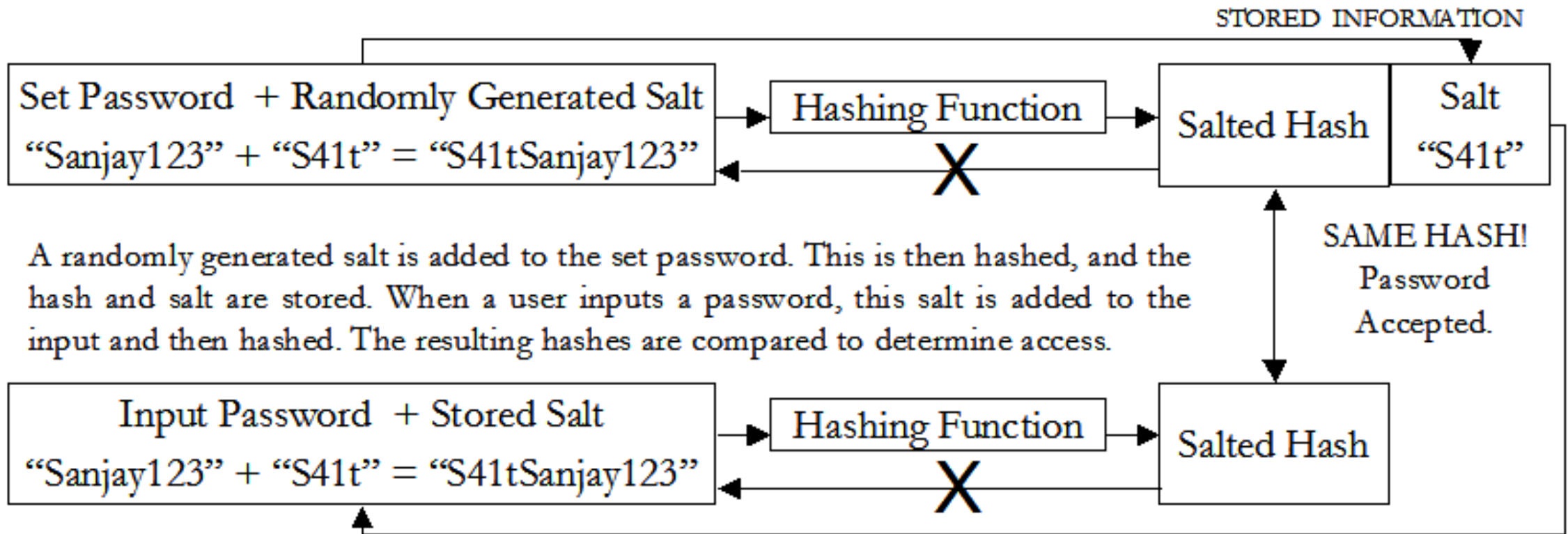


## Verification an input password against the stored hash





# Hashing Password - Salting





## Hashing Password – Salting Advantages



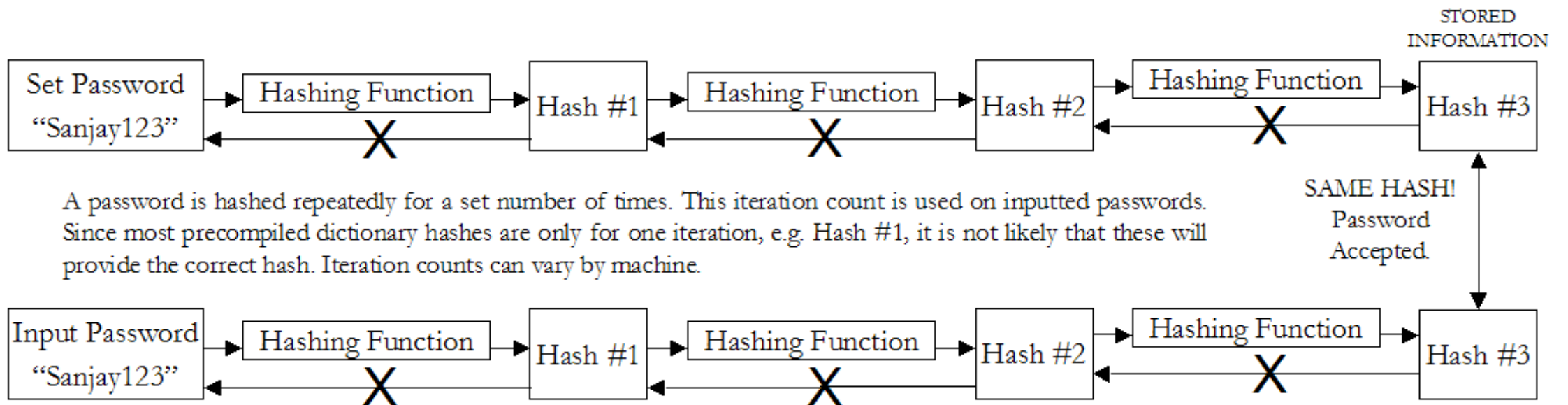
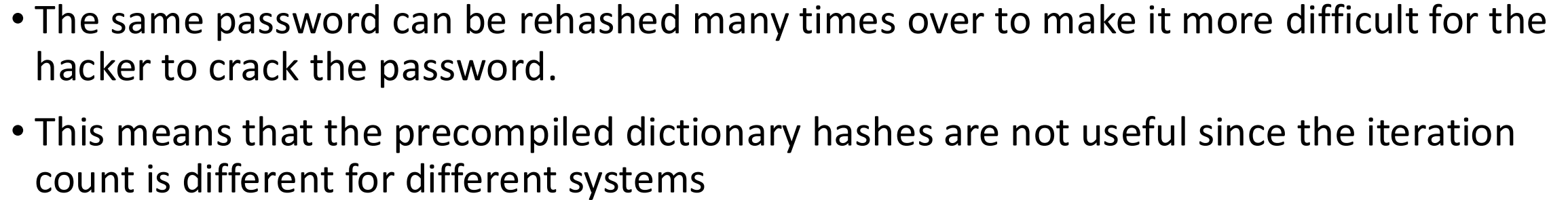
Without salt, attacker can pre-compute hashes of all dictionary words once for all password entries

- Same hash function on all systems
- Identical passwords hash to identical values; one table of hash values can be used for all password files

With salt, attacker must compute hashes of all dictionary words once for each password entry

- With 12-bit random salt, same password can hash to  $2^{12}$  different hash values
- Attacker must try all dictionary words for each salt value in the password file





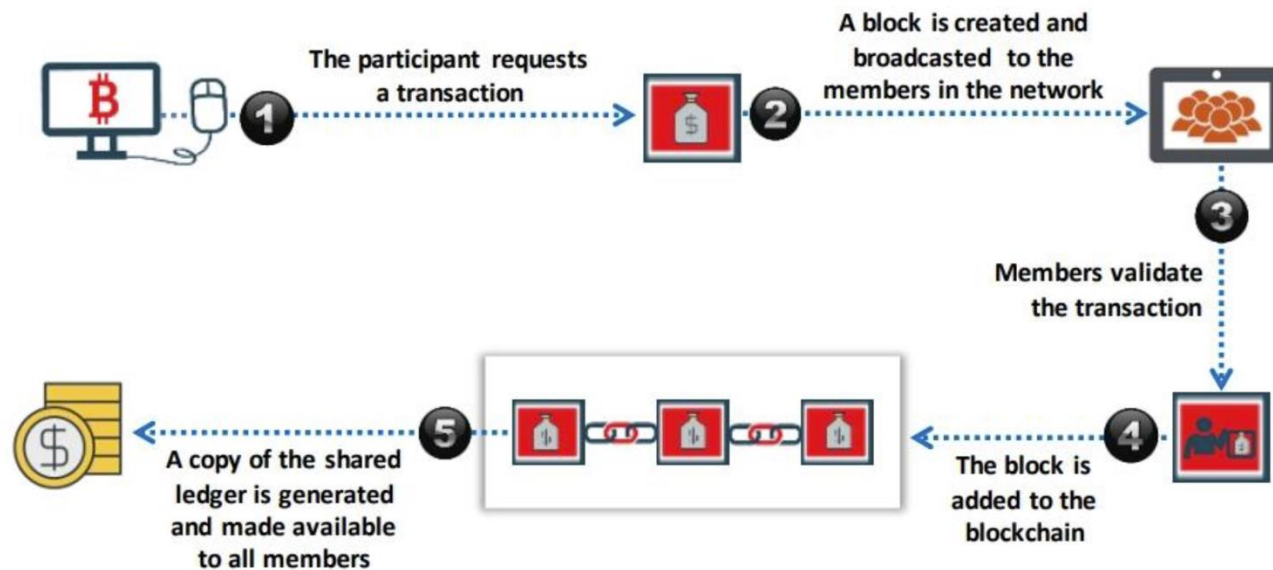


# Applications of Cryptography - Blockchain



- A blockchain, also referred to as **distributed ledger technology** (DLT), is used to record and store the history of transactions in the form of blocks
- For multiple transactions, **multiple blocks are created**, which are linked together to form a "blockchain"

## Process of Creating a Blockchain



## Types of Blockchain

- 1** Public Blockchain
- 2** Private Blockchain
- 3** Federated Blockchain
- 4** Hybrid Blockchain

# Do you have any questions?



[rc3.kaust.edu.sa](https://rc3.kaust.edu.sa)



Follow us @rc3kaust

